

FASHION
THAT **MOVES**
WITH YOU

S A B L E



TEAM MEMBERS | TEAM KAIZO

STUDEN1599841 MUHAMMAD MUJTABA HYDER
STUDEN1598139 ALI JAWAD
STUDEN1592376 MANAHIL KHAN
STUDEN1594885 ALISHBA ASGHAR

APBOT PROJECT

Faculty MUHAMMAD REHAN
KAPPAYA

Table of Contents

Contents are listed explicitly for compatibility across Microsoft Word viewers and protected-view previews.

Section	Title
	Document Control
1	Executive Summary
2	Background and Necessity
3	Proposed Solution
3.1	SABLE Visual Identity
3.2	ApBot Visual Identity
4	Purpose of This Document
5	Scope of the Project
6	Constraints
7	Objectives
8	Main Use Cases
9	Functional Requirements and Traceability
10	Non-Functional Requirements
11	System Architecture
12	Dialog Flow
13	AI and Machine Learning Design
14	Database Design
15	API and Interface Requirements
16	User Interface and Feature Description
17	Hardware and Software Requirements
17.1	Hardware
17.2	Software
18	Installation and Execution Instructions
18.1	Model and Data Initialization
19	Testing and Verification
19.1	Test Data
20	Assumptions
22	Future Enhancements
23	SRS Deliverables Checklist
24	Conclusion
A	Appendix A: Repository Structure

Document Control

This report describes the implemented academic prototype, maps the project to the supplied Software Requirements Specification, documents the architecture and data model, and records implementation boundaries honestly. Cart, checkout, payment, inventory, and tracking functions are treated as supporting demonstrations; the primary project objective remains the AI/ML conversational assistant.

1. Executive Summary

SABLE is a luxury-fashion website integrated with ApBot, an AI and ML conversational shopping assistant. The project combines a React/Vite frontend, a Node.js/Express orchestration backend, a Python/Flask TensorFlow service, NLTK preprocessing, a Keras intent-classification model, and MongoDB persistence. The assistant accepts natural-language text and browser voice input, predicts user intent, retrieves product or order information, and returns contextual responses and interface actions.

The implementation satisfies the central academic objective of demonstrating an AI/ML chatbot integrated with a website. The most important verified capabilities are intent classification, pattern-based training data, NLTK tokenization and lemmatization, Bag-of-Words vectorization, TensorFlow/Keras inference, REST API integration, product assistance, FAQ/support responses, and demonstration order guidance. The project is best described as an intent-classification assistant with deterministic commerce workflows rather than a production payment or logistics platform.

The repository contains source code, model artifacts, a Jupyter notebook, database schemas, seed scripts, architecture material, assumptions, and test-case documentation. The final audit verified the major callback, wishlist authorization, NLP spelling-correction, and Python service-fallback requirements. The remaining submission artifacts are the public Google Colab notebook link and working demonstration video link, which are included in the final deliverables.

2. Background and Necessity

Online shoppers often need help that is difficult to provide through static navigation alone. They may ask for a product using informal language, request a recommendation based on price or colour, ask about material or fit, or require order-related guidance. Traditional search bars and fixed FAQ pages do not maintain conversational context and can increase the effort required to reach a purchase decision.

Chatbots address this problem by providing quick, continuous assistance. In the proposed system, the assistant is not intended to replace the entire e-commerce platform. Its purpose is to understand a customer message, classify the user intent, retrieve relevant information, and guide the user through the next appropriate action. This interpretation is aligned with the SRS, which identifies full inventory, order-processing, customer-database, and payment-gateway integrations as beyond the core project scope [1].

3. Proposed Solution

ApBot uses a three-layer application design. The React frontend presents the website and conversational interface. The Node.js/Express backend acts as an orchestration and business-rule layer. It forwards user text to the Python service, resolves conversational context, queries MongoDB, and returns structured response data and frontend actions. The Python/Flask service performs NLP preprocessing and Keras model inference.

Layer	Technology	Primary responsibility
Presentation layer	React, Vite, GSAP, Lenis, browser Web Speech API	Website UI, ApBot panel, product cards, forms, voice input, and speech output
Application layer	Node.js, Express, Mongoose, JWT, bcryptjs	API routing, authentication, context resolution, product/order operations, and frontend action payloads
AI/ML layer	Python, Flask, TensorFlow, Keras, NLTK, NumPy	Intent classification, tokenization, lemmatization, vocabulary encoding, and prediction confidence
Data layer	MongoDB	Products, users, carts, orders, and subscribers

3.1 SABLE Visual Identity

The report follows the live SABLE website's visual language so that the documentation feels like part of the same product. The design is editorial, restrained, monochrome, and image-led: a warm bone background, deep ink typography, muted grey metadata, thin rules, oversized wordmarks, compact uppercase labels, and a restrained gold accent.

Brand token	Website value	Report usage
Bone	#EFEDE8	Page background, cover field, table highlights, and light panels
Ink	#101010	Primary headings, body emphasis, rules, and dark action elements
Mid	#8A8781	Metadata, captions, secondary labels, and supporting text
Gold accent	#C5A059	Hairlines, section accents, and selected cover labels
Primary typeface	Archivo	Body text, controls, navigation-style labels, tables, and captions
Luxury headings	Cinzel / Italiana	Section headings, editorial title treatment, and report hierarchy
Editorial treatment	Large wordmark + whitespace + fashion photography	Cover, visual references, and section hierarchy

SABLE

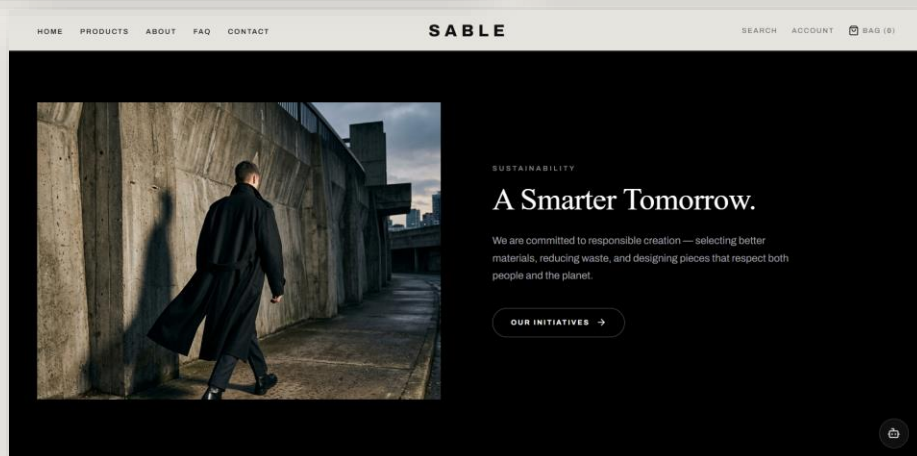
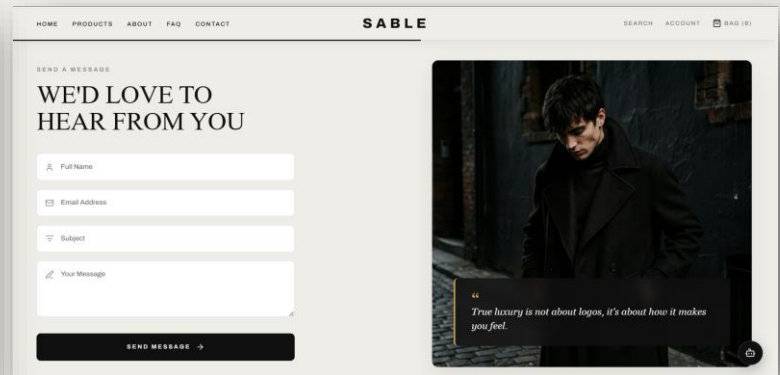
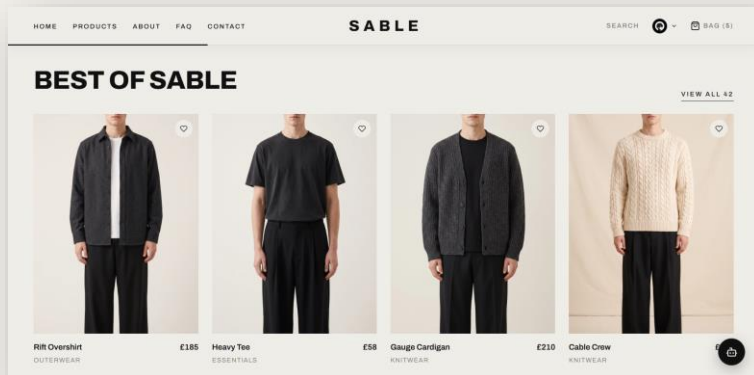




Figure 0. SABLE website visual reference: New season editorial image



Outerwear



Knitwear



Tailoring



Autumn 2026 lookbook



The Atelier

Website imagery source: SABLE public CDN assets referenced by MUJTABA

3.2 ApBot Visual Identity

ApBot uses the same SABLE interface language: the ink-and-bone palette, compact Archivo controls, thin line separators, and a restrained assistant mark. The live panel is included as a visual reference for the chatbot integration described in this report.

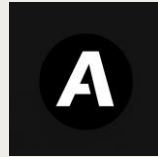


Figure 0A. ApBot assistant logo used by the SABLE website

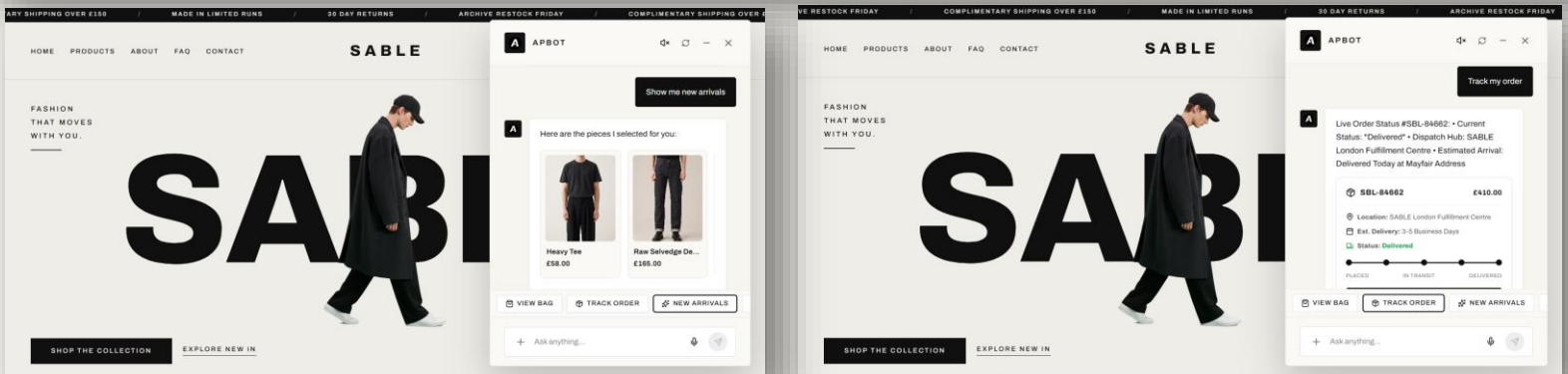
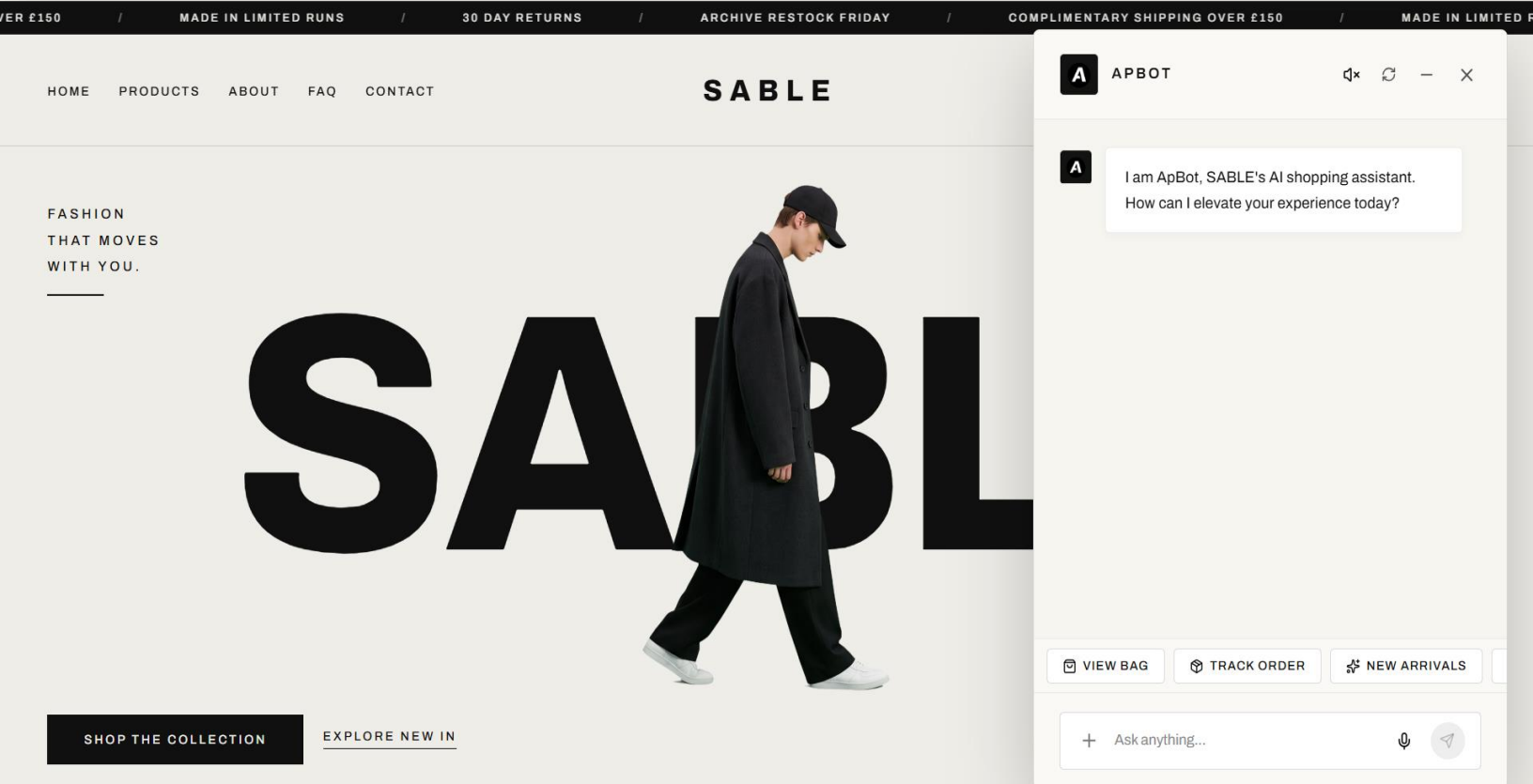


Figure 0B. ApBot panel visual reference from the live SABLE website

4. Purpose of This Document

This document provides a complete technical and academic description of ApBot. It is intended for evaluators, supervisors, developers, testers, and stakeholders who need to understand the problem, solution, system design, implementation evidence, installation process, testing boundaries, assumptions, and future enhancement path.

5. Scope of the Project

The core scope is the development and integration of an AI/ML e-commerce assistant. ApBot should welcome users, understand product-related questions, provide product information, offer recommendations, answer common support questions, assist with order tracking, and guide a user toward checkout. The website and MongoDB integration provide a realistic environment in which the chatbot can demonstrate these behaviors.

Full production e-commerce operations are outside the core scope. Real payment authorization, courier APIs, enterprise inventory synchronization, tax calculation, warehouse management, and customer-service ticketing are optional extensions rather than required AI/ML deliverables. The current cart and checkout flows are therefore documented as demonstrations.

In scope	Out of core scope
Intent classification and NLU/NLP processing	Real payment gateway and card authorization
Product information and database-backed search	Live courier or DHL tracking integration
Contextual product guidance and recommendations	Enterprise inventory and warehouse synchronization
FAQs, support responses, and fallback handling	Production ticketing, email, and CRM integration
Website chatbot and REST integration	General-purpose question answering
Demonstration cart, checkout, and tracking actions	Financial settlement, refunds, tax, and chargebacks

6. Constraints

Chatbot accuracy depends on the quality and coverage of the training patterns in intents.json. Natural language may contain slang, misspellings, ambiguous references, or requests outside the supported fashion domain. The current model is an intent classifier, so it does not provide unrestricted general knowledge reasoning.

Voice recognition depends on browser support, microphone permission, and device audio capability. MongoDB must be available for database-backed flows. The current visual-search entry point is a prototype that performs filename/keyword-assisted catalog matching rather than true image embedding search. Checkout is a demonstration flow and must not be presented as a real payment transaction.

7. Objectives

- Develop a conversational AI assistant using TensorFlow/Keras and a structured intent dataset.
- Implement NLTK tokenization, lemmatization, vocabulary creation, and binary Bag-of-Words encoding.
- Expose the trained model through a Flask REST API and integrate it with a Node/Express backend.
- Provide a user-friendly React website with contextual product assistance and support responses.
- Demonstrate product lookup, recommendation, order tracking guidance, and checkout redirection.
- Document the system architecture, assumptions, test data, installation process, and limitations.

8. Main Use Cases

Use case	Actor	Description	Expected result
UC-01: Product enquiry	Customer	Asks for a product, category, colour, or price range.	ApBot returns relevant product cards from MongoDB.
UC-02: Product details	Customer	Asks about the selected product's material, price, sizes, or availability.	ApBot retrieves and presents stored product information.
UC-03: Contextual follow-up	Customer	Refers to "the first one," "similar," or "cheaper options."	ApBot uses the previous conversation context.
UC-04: Support/FAQ	Customer	Asks about shipping, returns, payment methods, or contact support.	ApBot provides a documented support response.
UC-05: Order guidance	Customer	Provides an order/tracking ID or asks about delivery.	ApBot returns local/demo tracking information.
UC-06: Checkout guidance	Customer	Asks to proceed with purchase.	ApBot checks the cart and redirects to a demonstration checkout flow.
UC-07: Unsupported request	Customer	Asks an unrelated question.	ApBot politely redirects the conversation to SABLE services.

9. Functional Requirements and Traceability

The following matrix maps the supplied SRS requirements to the audited repository implementation. "Implemented" indicates that source evidence exists. "Partial" indicates a prototype or simplified implementation. "Not verified" indicates that a claim requires a reproducible test or additional evidence.

ID	Requirement	Evidence	Status
FR-01	Pattern matching through intents, patterns, and responses	intents.json, nlp.py	Implemented
FR-02	NLU intent, context, and expected response recognition	predictor.py and apbotRoutes.js	Partial: model intent plus deterministic context rules
FR-03	NLP tokenization and word vectorization	NLTK tokenizer, lemmatizer, and BoW encoder	Implemented
FR-04	TensorFlow/Keras chatbot model	train.py and chatbot_model.keras	Implemented
FR-05	Website chatbot interface	React ApBot component	Implemented
FR-06	REST API integration	Flask predict and Express message routes	Implemented
FR-07	Product details and similar alternatives	MongoDB Product queries	Implemented
FR-08	Order tracking	Order lookup and elapsed-time status logic	Partial: local/demo tracking
FR-09	FAQs and common issue support	Intent data and support responses	Implemented
FR-10	Checkout redirection	Checkout action, forms, and Order creation	Partial: mock checkout, no gateway
FR-11	Jupyter/Colab source deliverable	ApBot_Training.ipynb	Implemented
FR-12	Public GitHub access	Public repository	Implemented

10. Non-Functional Requirements

ID	Requirement	Implementation evidence	Status
NFR-01	Compatibility	React/Vite browser application with voice feature detection	Partial; voice is browser-dependent
NFR-02	Security	JWT, bcrypt, and protected operations exist	Implemented and verified
NFR-03	User experience	Responsive UI, animations, loading states, and fallback messages	Implemented at prototype level
NFR-04	Performance	Local model inference and lightweight API path	Not independently benchmarked
NFR-05	Reliability	Python-service fallback handling	Implemented and verified
NFR-06	Scalability	Decoupled frontend, backend, and AI service	Architecture supports it; load test not supplied

11. System Architecture

The user interacts with the React/Vite website. ApBot sends a message and the current conversation context to the Express backend. Express requests intent classification from the Flask AI service, then applies business rules and MongoDB queries before returning a structured response. The structured response may contain natural-language text, product/order data, frontend actions, and updated conversation context.

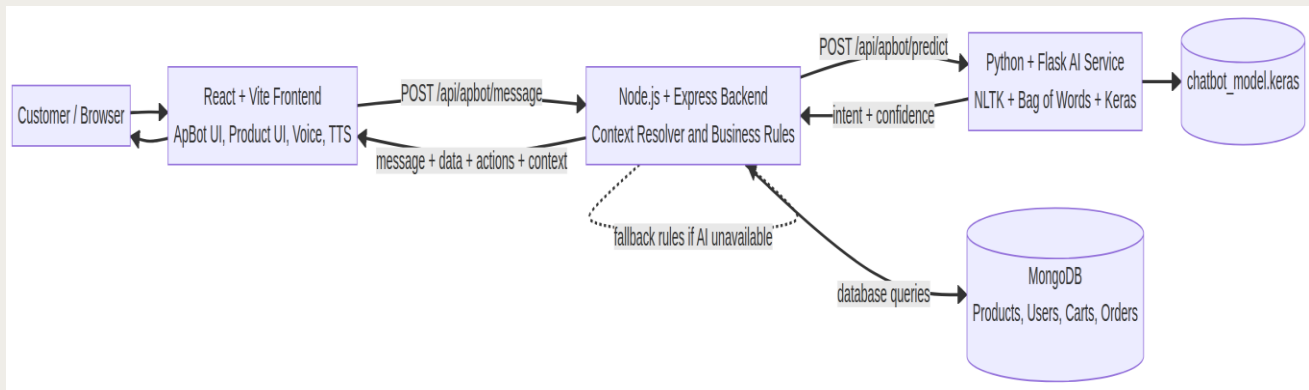


Figure 1. ApBot system architecture and service boundaries

The architecture separates model inference from application orchestration. This makes the AI layer independently testable and allows the Node service to provide deterministic fallback behavior when the Python service is unavailable. The architecture is appropriate for an academic demonstration of AI/ML integration.

12. Dialog Flow

The dialog flow begins with a welcome message, accepts text or browser voice input, predicts an intent, resolves context and database information, and returns a response. The assistant can continue a multi-turn conversation or end with a fallback response when the request is outside the supported SABLE domain.

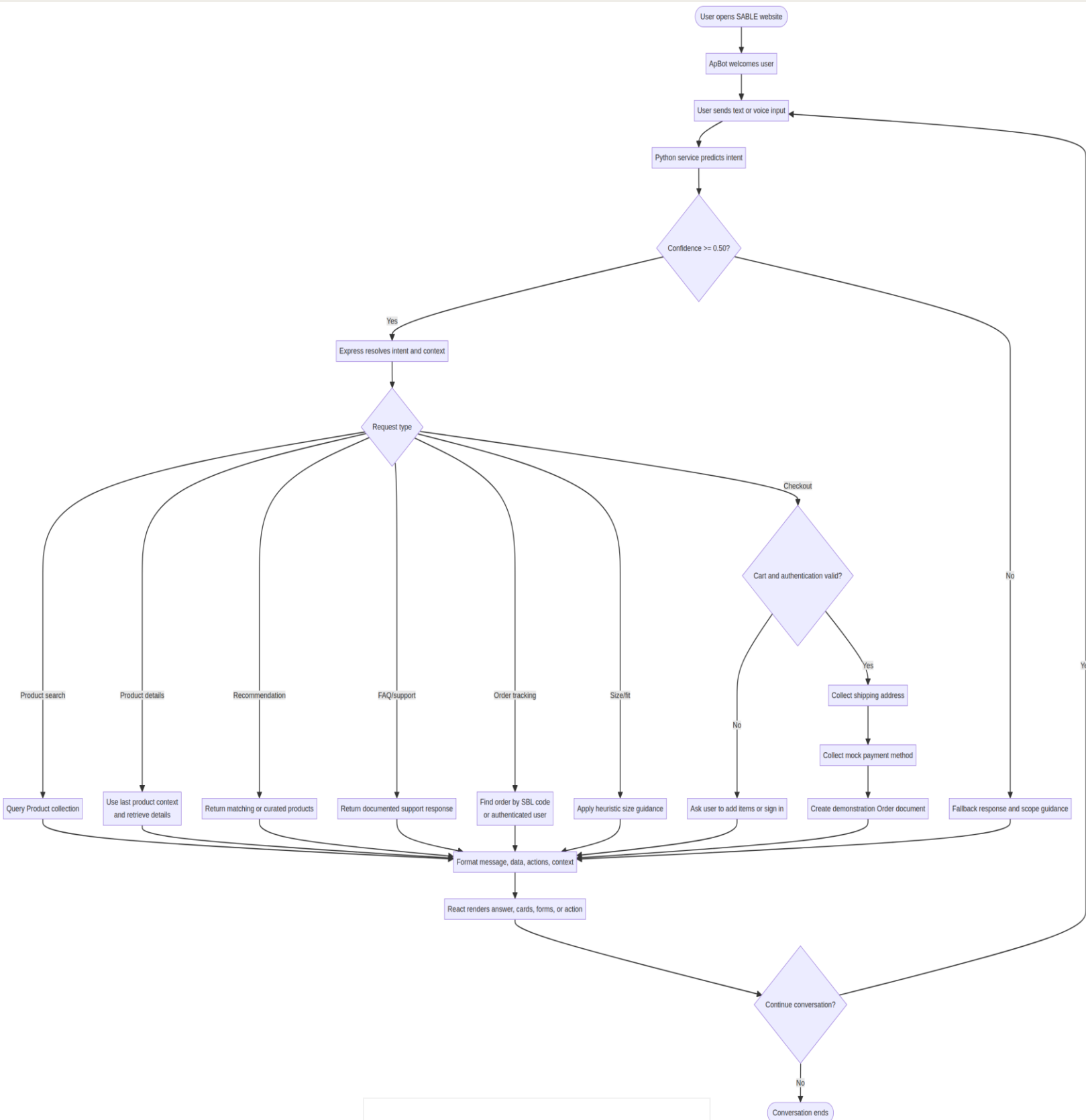


Figure 2. ApBot conversation and action flow

13. AI and Machine Learning Design

The AI component is an intent-classification chatbot. The training data is stored in `intents.json` and contains intent tags, example patterns, and response alternatives. The training script processes the patterns and generates the model artifacts required during runtime.

Pipeline stage	Implementation
Training data	Intent tags, example user patterns, and response templates in <code>apbot/data/intents.json</code>
Tokenization	NLTK word tokenization converts each sentence into tokens.
Normalization	WordNet lemmatization and punctuation filtering normalize tokens.
Vectorization	Each sentence is represented as a binary Bag-of-Words vector against the vocabulary.
Model	Keras Sequential network with Dense, Dropout, Dense, Dropout, and Softmax output layers.
Training	Stochastic Gradient Descent with momentum, categorical cross-entropy, batch size 5, and 200 epochs.
Runtime	Flask receives a message and returns an intent, confidence, response, and context payload.
Business resolution	Express applies product filters, context rules, order lookup, and UI action selection.

The model should be described as a supervised intent classifier. Product retrieval, context references, cart actions, checkout state, and order tracking are primarily resolved by deterministic Node.js logic after the model returns an intent. This distinction is important for an accurate academic report.

14. Database Design

MongoDB stores the application documents in the `sable` database. The main collections are `Product`, `User`, `Cart`, `Order`, and `Subscriber`. Cart and order items are embedded snapshots containing product identifiers and display fields. User-to-order and user-to-cart relationships are logical associations through email strings rather than strict `ObjectId` foreign keys.

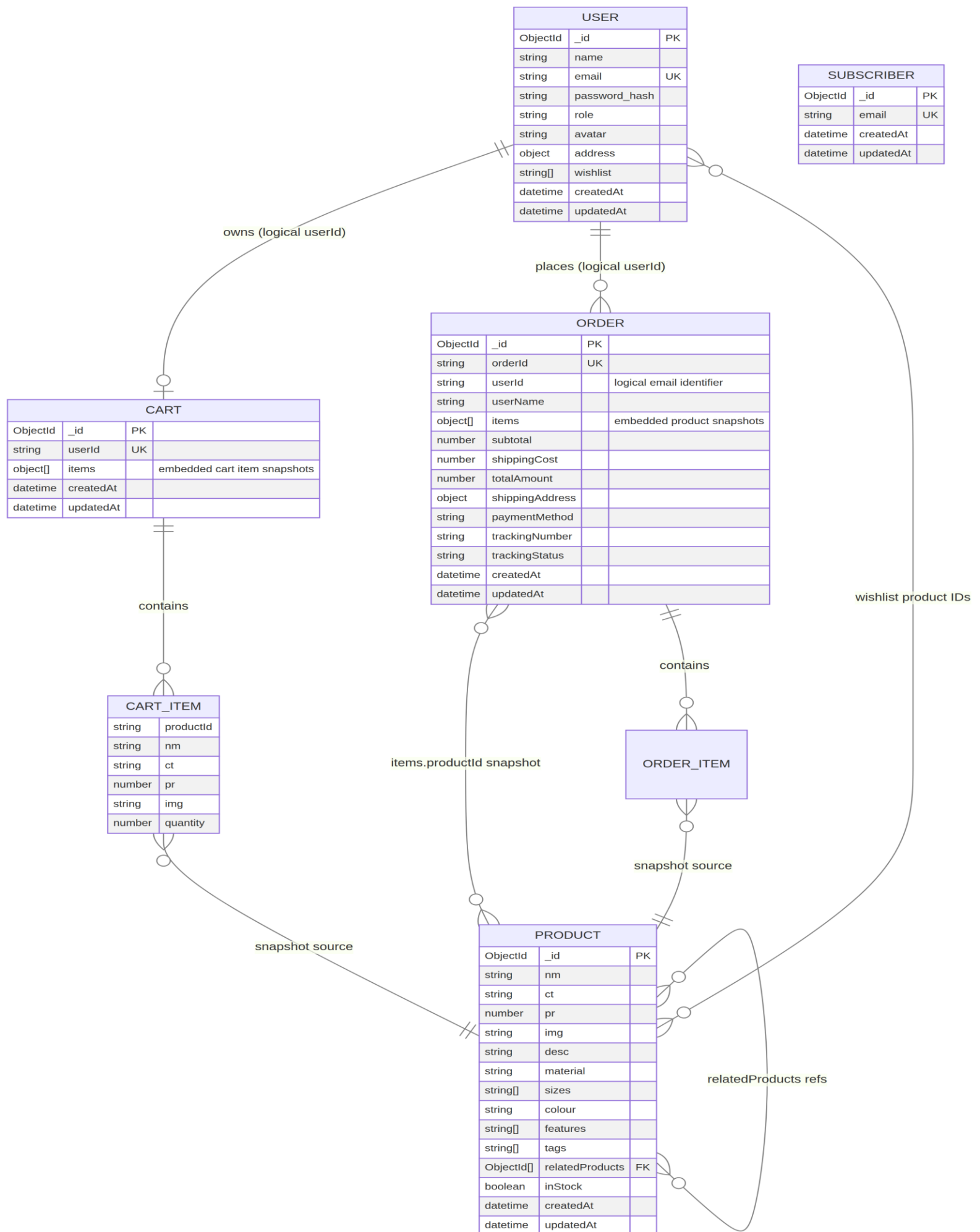


Figure 3. MongoDB entity and logical relationship diagram

Collection	Main fields	Purpose
Product	nm, ct, pr, img, desc, material, sizes, colour, features, tags, relatedProducts, inStock	Source of truth for catalog information and product responses.
User	name, email, password, role, avatar, address, wishlist, timestamps	Authentication, profile, address, and wishlist data.
Cart	userId, embedded items, quantity, product snapshot fields	Demonstration/persistent cart representation for a logical user.
Order	orderId, userId, userName, items, totals, address, payment label, tracking fields	Demonstration order creation and local tracking.
Subscriber	email, timestamps	Newsletter subscription records.

The editable database diagram source is supplied with this report as assets/apbot_db_schema.mmd. The diagram deliberately labels logical relationships and embedded snapshots so that the design is not misrepresented as a fully normalized relational schema.

15. API and Interface Requirements

Method	Endpoint	Purpose	Access
GET	/api/health	Backend health response	Public
GET	/api/products	List products	Public
GET	/api/products/:id	Product details	Public
POST	/api/auth/register	Register and issue JWT	Public
POST	/api/auth/login	Authenticate and issue JWT	Public
POST	/api/apbot/message	Process message and return response/actions/context	Optional bearer token
POST	/api/apbot/predict	Python intent prediction	Internal AI service
GET	/api/cart/:userId	Retrieve/create cart	Optional token
POST	/api/cart/save	Save cart items	Optional token
POST	/api/orders/create	Create demonstration order	Current implementation is optional-token
GET	/api/orders/track/:code	Find order by tracking code	Public demo lookup
PUT	/api/orders/update-status	Update order status	JWT + Admin role required
POST	/api/user/address	Update authenticated address	JWT required
GET	/api/user/wishlist	Read wishlist	JWT required
POST	/api/user/wishlist/toggle	Add/remove wishlist item	JWT required
POST	/api/subscribers/subscribe	Newsletter subscription	Public

16. User Interface and Feature Description

Feature	Description	SRS relevance
Welcome assistant	ApBot greets the visitor and explains supported assistance.	Supports conversational interface.
Natural-language search	Users ask for products by category, colour, keyword, or price.	Core product-assistance requirement.
Contextual references	The assistant can resolve references such as “the first one” from recent results.	Demonstrates context handling.
Product cards	Product name, price, image, and detail actions are rendered inside chat.	Improves website integration.
Voice input and TTS	Browser Speech Recognition and speech synthesis are used when supported.	Optional extension to text chatbot.
FAQ/support	Hardcoded domain responses cover shipping, returns, payment, and contact requests.	Core support requirement.
Checkout demonstration	Address and mock payment steps produce an order document.	Demonstrates checkout redirection without real payment.
Order tracking demonstration	Order IDs can be looked up and shown with local status progression.	Core tracking guidance at demo level.

17. Hardware and Software Requirements

17.1 Hardware

Component	Minimum recommendation
Processor	Intel Core i5 or equivalent
Memory	8 GB RAM or higher
Storage	500 GB hard disk or equivalent available storage
Display/input	Colour display, keyboard, and mouse
Network	Internet access for package installation and repository access

17.2 Software

Software area	Requirement
Frontend	React/Vite, JavaScript, HTML5/CSS
Backend	Node.js, Express.js, Mongoose, JWT, bcryptjs
AI/ML	Python 3.10+, TensorFlow, Keras, NLTK, NumPy, Flask
Data store	MongoDB; JSON intent data and model artifacts
Development tools	Git, Node Package Manager, Python virtual environment, Jupyter Notebook
Browser	Modern Chromium, Edge, Firefox, or Safari; voice requires browser support

18. Installation and Execution Instructions

The following procedure is the recommended reproducible setup. MongoDB should be running before the backend is started. The frontend API base URL is currently configured for local development.

1. Clone the repository: `git clone https://github.com/MujtabaZadai/APBOT_E-commerce.git`
2. Enter the project directory: `cd APBOT_E-commerce`
3. Install frontend dependencies: `npm --prefix client install`
4. Install backend dependencies: `npm --prefix server install`
5. Create the server environment file: copy `server/.env.example` to `server/.env` and set `MONGO_URI` and `JWT_SECRET`.
6. Create and activate a Python virtual environment inside `apbot`.
7. Install Python requirements with `pip install -r requirements.txt`.
8. Download required NLTK resources if the environment does not already contain them.
9. Run `node server/seed.js` only when it is acceptable to initialize/reset development Product seed data.
10. Start the Python service with `python apbot/app.py`, the Node service with `node server/server.js`, and the frontend with `npm --prefix client run dev`.
11. Open the Vite URL displayed by the frontend terminal and test the ApBot panel.

18.1 Model and Data Initialization

The repository includes `chatbot_model.keras`, `words.pkl`, and `classes.pkl`, so normal runtime operation does not require retraining. If the intent dataset is changed, run `python apbot/train.py` to regenerate the model artifacts. The seed script is intended for development and should not be used against a shared production database without review because seed scripts may replace existing Product documents.

19. Testing and Verification

Testing covers source-level verification, build verification, documented conversational scenarios, NLP unit testing, and service-fallback integration testing. The project contains 24 documented conversational scenarios covering greetings, natural conversation, product search, price filtering, product details, cart and wishlist actions, FAQs, order tracking, fallback behavior, and service-failure handling. Executable automated verification currently includes:

- 16/16 Domain Spell Corrector NLP unit tests passed: `python apbot/test_spelling.py`
- TC-13 Python/Flask service fallback integration test passed: `node server/test_tc13_fallback.js` or `npm run test:tc13`

The 24 conversational scenarios are maintained as the documented functional test matrix and should not be described as 24 automated integration tests.

Test ID	Scenario	Expected behavior	Status/evidence
---------	----------	-------------------	-----------------

TC-01	Greeting	ApBot welcomes the user.	Documented pass in existing test matrix
TC-02	Natural conversation	Friendly domain response.	Documented pass
TC-03	Product search	Returns database-backed product cards.	Documented pass
TC-04	Price filter	Preserves context and filters by price.	Documented pass
TC-05	Context index	“First one” resolves to recent result.	Documented pass
TC-06	Product details	Returns material, sizes, price, and description.	Documented pass
TC-07	Similar product	Returns same-category alternatives.	Documented pass
TC-08	Cart action	Returns add-to-cart action.	Verified PASS after implementing callback fix and rerunning test
TC-09	Wishlist action	Toggles wishlist for authenticated user.	Verified PASS after authenticated route verification
TC-10	FAQ/support	Returns shipping/returns/contact response.	Documented pass
TC-11	Order tracking	Returns local/demo status response.	Documented pass at demo level
TC-12	Out-of-scope query	Polite fallback keeps domain scope.	Documented pass
TC-13	Python service unavailable	Fallback response without UI crash.	Verified PASS Python/Flask service unavailable fallback tested successfully with HTTP 200 and valid fallback response.
TC-14	Frontend production build	Vite produces a dist bundle.	Verified PASS during audit
TC-15	Backend syntax	Node route/model files parse.	Verified PASS during audit
TC-16	Python syntax	Flask/training/chatbot modules compile.	Verified PASS during audit

19.1 Test Data

The primary chatbot test data is stored in `apbot/data/intents.json`. Product test data is supplied through the MongoDB seed and migration scripts. Test messages include greetings, product categories, colours, price ranges, product references, FAQ terms, order identifiers, sizing terms, checkout terms, and unrelated questions for fallback behavior.

20. Assumptions

Area	Assumption
Runtime	MongoDB is available locally or through a configured <code>MONGO_URI</code> .
Model	Committed model artifacts correspond to the committed intents dataset.
Voice	The browser supports Speech Recognition and <code>speechSynthesis</code> , and the user grants microphone permission.
Product source	MongoDB Product documents are the source of truth for product display data.
Checkout	Checkout is a demonstration workflow and does not authorize or settle a real payment.
Tracking	Order tracking is a local time-based simulation unless a carrier integration is added.
Users	JWT and <code>bcrypt</code> support account flows. Production deployment still requires secure environment-based secret management, HTTPS, rate limiting, validation, logging, and monitoring.
Testing	Documented test cases must be rerun after callback and security fixes.

21. Future Enhancements

- Expand automated test coverage and add CI workflow for continuous verification.
- Replace deterministic product rules with a validated recommendation or retrieval model.
- Add a real image-upload and visual-embedding pipeline only if visual AI becomes a project extension.
- Implement a validated sizing model using documented size charts and anonymized test data.
- Add persistent support tickets and notifications if customer-service workflows are required.
- Use environment-based API URLs, HTTPS, secret management, rate limiting, validation, logging, and monitoring for deployment.
- Integrate a payment provider only as a separately scoped extension and never store raw card data.

22. SRS Deliverables Checklist

Required deliverable	Included or action required
Problem definition	Included in Sections 2 and 4.
Design specifications	Included in architecture, AI/ML, database, UI, and API sections.
Dialog flow diagram	Included as Figure 2 and editable Mermaid source.
System architecture diagram	Included as Figure 1 and editable Mermaid source.
Database diagram	Included as Figure 3 and editable Mermaid source.
Source code	Public GitHub repository supplied in the cover page and references.
Jupyter notebook	Present in apbot/notebooks/ApBot_Training.ipynb.
Test data	intents.json and product seed data documented in Section 19.
Installation instructions	Included in Section 18.
Website integrated with chatbot	Implemented through React, Express, and Flask integration.
Assumptions	Included in Section 20.
Public GitHub link	Included.
Working video clip	Included

23. Conclusion

ApBot demonstrates the integration of artificial intelligence, machine learning, natural-language processing, and web engineering in an e-commerce assistance scenario. The project includes the principal SRS elements: a structured intent dataset, NLTK preprocessing, Bag-of-Words representation, TensorFlow/Keras model training, a Flask REST service, a website chatbot, product assistance, support responses, and order guidance.

The project should be evaluated primarily as an AI/ML chatbot rather than as a complete commercial e-commerce system. Cart, checkout, payment, inventory, visual search, and tracking features should remain lightweight demonstrations unless the project brief is explicitly expanded. With the callback and authorization issues corrected, reproducible test evidence established, scope claims accurately documented, and the required Colab notebook and working demonstration video supplied, the submission presents a coherent and defensible solution aligned with the SRS.

Recommended final project description: “ApBot is a TensorFlow/Keras intent-classification e-commerce assistant that uses NLTK preprocessing and Bag-of-Words vectors for intent prediction, while deterministic Node.js business rules provide product guidance, contextual assistance, FAQs, demonstration order tracking, and checkout redirection.”

Appendix A: Repository Structure

Path	Role
apbot/	Python Flask AI service, training script, chatbot modules, intents, model artifacts, and notebook.
client/	React/Vite website and ApBot interface.
server/	Node/Express API, routes, middleware, Mongoose models, seed and migration scripts.
docs/	SRS compliance and capability notes.
submission/	Academic report, assumptions, architecture, dialog flow, test cases, and submission README.
final_documentation/	This complete report, traceability matrix, audit memo, and diagram assets.

THE END